

De semáforos a monitores

Programación Concurrente y Paralela, Clase práctica 3

Segundo cuatrimestre 2026

Nota. Este es el hand-out de una clase que no llegamos a dar en vivo. Está escrito para leerse como si fuera la clase: con las mismas vueltas, las mismas preguntas que uno se haría en el pizarrón, y el mismo orden en que iríamos construyendo las ideas. Si en algún punto dice “pensemos” o “fijense”, es literal: conviene parar un segundo antes de seguir leyendo.

Bien, arranquemos. La clase pasada resolvimos un par de problemas con semáforos. Hoy vamos a usarlos de punto de partida para construir, y armar un monitor desde 0. ¿Para qué, si ya existe? Shhh, ustedes confíen.

1. Repaso: el desvío de Saldungaray

Antes de meternos con monitores, volvamos al segundo problema de la práctica pasada. El problema era el desvío de tránsito de Saldungaray (los *Zorros grises de Saldungaray*): un solo sentido habilitado por vez, varios autos del mismo sentido cruzando juntos, pero ni uno del sentido contrario mientras haya alguien adentro. Es el mismo patrón de molinete más interruptor que vimos para lectores-escritores, sólo que ahora simétrico entre dos clases en vez de dar prioridad a una.

1.1. Parte 1: CaminoUnaVia.java

```
public void entrar(int direction) throws InterruptedException {
    turnstile.acquire();
    countMutex[direction].acquire();
    count[direction]++;
    if (count[direction] == 1) {
        resource.acquire();
    }
    countMutex[direction].release();
    turnstile.release();
}

public void salir(int direction) throws InterruptedException {
    countMutex[direction].acquire();
    count[direction]--;
    if (count[direction] == 0) {
        resource.release();
    }
    countMutex[direction].release();
}
```

Primero entendamos por qué esto funciona, sin apurarnos con el tema de starvation. **resource** es el interruptor: el primer auto de un sentido lo toma (apaga la luz del sentido contrario) y el último en salir lo libera. Mientras haya un auto de un sentido adentro, `count[direction]` no vuelve a 0, así que **resource** sigue tomado por ese sentido. Con eso ya tenemos exclusión mutua entre sentidos: mientras A esté cruzando, ningún auto de B puede tomar **resource**.

turnstile cumple otro rol, más sutil. Se retiene durante todo **entrar**, inclusive mientras el primer auto de un sentido está bloqueado en **resource.acquire()** esperando a que el sentido contrario se vacíe. Eso es a propósito: mientras ese primer auto lo retiene, nadie más (de ningún sentido) puede pasar el molinete, así que el sentido que ya estaba activo termina de vaciarse en paz, sin que sigan entrando más autos de su propio sentido a último momento.

Ahora sí, veamos qué puede salir mal. Supongamos que **turnstile** es un semáforo común de Java, `new Semaphore(1)`, sin el segundo parámetro (defaultea a false). Un semáforo *débil*: cuando se libera, cualquiera que esté pidiendo el permiso en ese momento se lo puede llevar, no necesariamente el que hace más tiempo que espera (no hay un orden). Sigamos una traza concreta para ver dónde se rompe:

1. El sentido A lleva un buen rato activo, con tránsito continuo: siempre hay algún auto A cruzando, `count[A]` nunca llega a 0.
2. Llega B1, que quiere cruzar en sentido B. Hace **turnstile.acquire()** y queda esperando en la cola interna del semáforo (en ese instante el molinete estaba ocupado por el registro de otro auto).
3. **turnstile** se libera. En simultáneo llega un auto nuevo, A6, que también llama a **turnstile.acquire()**. Como el semáforo es débil, Java no tiene ninguna obligación de atender primero a B1, que ya estaba en la cola: A6 puede colarse (esto se llama *barging*) y ganar el permiso.
4. A6 hace su registro (`count[A]` ya era mayor a 0, así que ni siquiera necesita tocar **resource**) y libera **turnstile** enseguida. B1 sigue esperando.
5. **turnstile** se libera otra vez. Llega A7. Se repite exactamente el paso 3: nada le impide a A7 colarse y ganarle el permiso a B1, que sigue esperando desde el paso 2.

Y así puede seguir indefinidamente: mientras siga llegando tráfico de A y cada auto nuevo tenga la chance de ganarle la carrera a B1 apenas **turnstile** se libera, no hay nada que obligue a que en algún momento le toque a él. El puente nunca deja de tener tránsito (A6, A7, A8... cruzan sin parar), pero B1 se queda esperando el molinete para siempre. Fíjense que el problema no está en **resource** ni en la lógica de `count[]`: está pura y exclusivamente en que **turnstile**, al ser débil, no le da ninguna prioridad a quien ya estaba haciendo cola.

La solución es hacer **turnstile** fuerte: `new Semaphore(1, true)`. Con el segundo parámetro en **true**, Java desactiva el *barging*: nadie que recién está llamando a **acquire()** le puede ganar el permiso a alguien que ya estaba en la cola, sin importar cuándo haya llegado. Volvamos a la traza: en el paso 3, A6 ya no podría colarse. B1, que llegó primero, es quien recibe el permiso apenas se libera. Lo mismo en cualquier ronda siguiente: un auto que ya estaba esperando el molinete siempre lo consigue antes que uno que llegó después, sin excepciones. Eso acota el problema: la cantidad de autos que pueden pasar antes que uno que ya está esperando queda fija en el momento en que se pone en la fila (a lo sumo, los que ya estaban delante de él), y como cada uno retiene el molinete a lo sumo lo que tarda en registrarse, un tiempo acotado, la espera total de cualquiera también queda acotada. Nadie espera para siempre.

Nota. El archivo de soluciones declara **turnstile** como `new Semaphore(1)`, sin el **true**. La traza

de arriba (la que realmente prueba que nadie espera para siempre) necesita la versión fuerte. En la práctica, un semáforo débil de Java se comporta razonablemente bien bajo carga pareja, porque el *barging* es poco frecuente cuando no hay un flujo adversarial como el del ejemplo; pero formalmente no da esa garantía. Tenerlo presente si piden justificar no starvation con precisión (Ejem, un parcial, ejem).

1.2. Parte 2: CaminoUnaViaFIFO.java

Ahora el puente soporta hasta `BRIDGE_CAPACITY` autos juntos, y se agrega una restricción: un auto que entró después no puede salir antes que uno que entró antes, no se pasan entre sí. `turnstile` y `resource` sólo deciden qué sentido cruza, no alcanzan. Hace falta un mecanismo aparte que ordene las salidas.

```
public void entrar(int direction) throws InterruptedException {
    // ... turnstile, countMutex, resource: igual que en la Parte 1 ...

    spots.acquire();

    queueMutex.acquire();
    int slot = nextSpot;
    nextSpot = (nextSpot + 1) % BRIDGE_CAPACITY;
    boolean esElPrimero = queued == 0;
    queued++;
    queueMutex.release();

    if (esElPrimero) {
        exitQueue[slot].release();
    }
    Auto.actual().miSlot = slot;
}

public void salir(int direction) throws InterruptedException {
    exitQueue[Auto.actual().miSlot].acquire();

    // ... countMutex, resource: igual que en la Parte 1 ...

    queueMutex.acquire();
    head = (head + 1) % BRIDGE_CAPACITY;
    queued--;
    boolean alguienQueda = queued > 0;
    int nuevoHead = head;
    queueMutex.release();

    if (alguienQueda) {
        exitQueue[nuevoHead].release();
    }
    spots.release();
}
```

La estrategia se llama *paso de testigo* (*baton passing*): en vez de pedirle a un semáforo compartido que sea fuerte, se arma a mano una fila de `BRIDGE_CAPACITY` semáforos, uno por lugar del puente,

todos arrancando cerrados. Cada auto que entra reserva el próximo lugar bajo `queueMutex`, en el orden en que llegó (`nextSpot` sólo avanza). El único que se habilita a sí mismo es el que encuentra la fila vacía. Al salir, un auto no libera cualquier candado: libera puntualmente el del lugar siguiente, pasándole el testigo a quien entró justo después.

¿Por qué esto respeta el orden de llegada sin que ningún semáforo individual sea fuerte? Vale el invariante: en todo momento hay a lo sumo un candado de `exitQueue` abierto entre los autos presentes, y es siempre el del más antiguo. Vale al principio (el primero que encuentra la fila vacía abre el suyo). Se mantiene con cada `entrar` (un auto nuevo se suma al final, no toca candados ajenos). Y se mantiene con cada `salir` (quien sale ya tenía el candado más antiguo abierto, y al irse abre el del que entró justo después). Como el invariante nunca se rompe, el único que puede completar `salir` en cualquier instante es el más antiguo entre los presentes.

El detalle clave: *nadie espera en un semáforo compartido*. Cada auto tiene el suyo, y quien le toca lo libera puntualmente. Esa idea (una cola de semáforos privados, uno por thread que espera) es lo que vamos a generalizar para construir un monitor reutilizable. Antes, la estiramos en dos direcciones: qué pasa si cambia el enunciado, y qué pasa si la escribimos más orientada a objetos.

1.3. Parte 2, sin capacidad máxima: una cola de verdad

Cambiamos el enunciado: el desvío ya no tiene límite de autos circulando (no hay K), sólo se mantiene que un auto que entró después no sale antes que uno que entró antes. El array circular de `exitQueue/head/nextSpot/queued` estaba atado a `BRIDGE_CAPACITY` porque necesitaba un tamaño fijo. Sin cota, ya no hay ningún número natural para ese tamaño: hace falta una cola de verdad, que crezca y decrezca con cada auto.

```
private final Semaphore queueMutex = new Semaphore(1);
private final Deque<Semaphore> exitQueue = new ArrayDeque<>();

public void entrar(int direction) throws InterruptedException {
    // ... turnstile, countMutex, resource: igual que antes ...

    queueMutex.acquire();
    Semaphore miTurno = new Semaphore(0);
    boolean esElPrimero = exitQueue.isEmpty();
    exitQueue.addLast(miTurno);
    queueMutex.release();

    if (esElPrimero) {
        miTurno.release();
    }
    Auto.actual().miTurno = miTurno;
}

public void salir(int direction) throws InterruptedException {
    Auto.actual().miTurno.acquire();

    // ... countMutex, resource: igual que antes ...

    queueMutex.acquire();
    exitQueue.removeFirst();
    Semaphore siguiente = exitQueue.peekFirst();
```

```

        queueMutex.release();

        if (siguiente != null) {
            siguiente.release();
        }
    }
}

```

La idea es la misma que el paso de testigo: cada auto crea su propio semáforo en 0 y se anota al final de la cola bajo `queueMutex`; si la encuentra vacía se abre a sí mismo. Al salir, se saca de la cabeza y le pasa el testigo a quien haya quedado al frente (`peekFirst()`). El invariante de antes sigue valiendo sin cambios, sólo que ahora `addLast/removeFirst/peekFirst` hacen el trabajo que antes hacían `head` y `nextSpot` módulo K. Y como ya no hay cota de concurrencia, tampoco hace falta `spots`.

Nota. Esta cola de semáforos privados, con un mutex aparte que sólo protege el armado y desarmado, es *exactamente* la forma que va a tener `esperando` en `MonitorCasero` más adelante. No es coincidencia: es la misma técnica, generalizada.

1.4. Una versión más objetosa: Lightswitch y FilaDeSalida

Incluso sin capacidad, la clase mezcla dos mecanismos independientes: quién puede cruzar (el interruptor de luz por sentido) y en qué orden salen (la cola de testigos). Nada impide darle nombre propio a cada uno y sacarlo a su propia clase, chica y reutilizable, en vez de tener cinco semáforos sueltos adentro de `CaminoUnaVia`.

El interruptor de luz generalizado, parametrizado por qué semáforo prende y apaga:

```

public class Lightswitch {
    private int contador = 0;
    private final Semaphore mutex = new Semaphore(1);
    private final Semaphore recurso;

    public Lightswitch(Semaphore recurso) {
        this.recurso = recurso;
    }

    public void encender() throws InterruptedException {
        mutex.acquire();
        contador++;
        if (contador == 1) {
            recurso.acquire();
        }
        mutex.release();
    }

    public void apagar() throws InterruptedException {
        mutex.acquire();
        contador--;
        if (contador == 0) {
            recurso.release();
        }
        mutex.release();
    }
}

```

```
}
```

Es el patrón *Lightswitch* del *Little Book of Semaphores*: el primero que entra prende la luz (toma recurso) y el último que sale la apaga, cualquier cantidad puede estar adentro mientras tanto. En *CaminoUnaVia* hace falta uno por sentido, compartiendo el mismo **resource**.

La cola de testigos, extraída tal cual estaba en la sección anterior:

```
public class FilaDeSalida {
    private final Semaphore mutex = new Semaphore(1);
    private final Deque<Semaphore> fila = new ArrayDeque<>();

    public Semaphore anotarse() throws InterruptedException {
        mutex.acquire();
        Semaphore miTurno = new Semaphore(0);
        boolean esElPrimero = fila.isEmpty();
        fila.addLast(miTurno);
        mutex.release();

        if (esElPrimero) {
            miTurno.release();
        }
        return miTurno;
    }

    public void avisarAlSiguiente() throws InterruptedException {
        mutex.acquire();
        fila.removeFirst();
        Semaphore proximo = fila.peekFirst();
        mutex.release();

        if (proximo != null) {
            proximo.release();
        }
    }
}
```

Y con las dos piezas nombradas, la clase que orquesta el problema queda casi como el enunciado en castellano:

```
private final Semaphore turnstile = new Semaphore(1);
private final Semaphore resource = new Semaphore(1);
private final Lightswitch[] lightswitch =
    { new Lightswitch(resource), new Lightswitch(resource) };
private final FilaDeSalida filaDeSalida = new FilaDeSalida();

public void entrar(int direction) throws InterruptedException {
    turnstile.acquire();
    lightswitch[direction].encender();
    turnstile.release();

    Auto.actual().miTurno = filaDeSalida.anotarse();
}
```

```
public void salir(int direction) throws InterruptedException {
    Auto.actual().miTurno.acquire();
    lightswitch[direction].apagar();
    filaDeSalida.avisarAlSiguiente();
}
```

`CaminoUnaVia` no cambió su comportamiento, sólo pasó de seis campos sueltos a cuatro colaboradores con nombre propio, cada uno responsable de una sola cosa. Es buena práctica de objetos aplicada a primitivas de sincronización, pero ninguna de las dos clases nuevas resuelve el problema de fondo: adentro de `Lightswitch` y `FilaDeSalida` seguimos pidiendo y liberando semáforos a mano. Cada problema nuevo con forma de cola necesitaría su propia clase igual de artesanal. Falta un mecanismo genérico del que cualquier espera se arme sin repetir el mutex-más-cola cada vez. A eso vamos ahora.

2. De goto a if/while: para qué sirve un monitor

¿De qué `goto` me hablan estos tipos? En programación estructurada es la primitiva de control más general que existe: con `goto` y etiquetas se puede construir cualquier `if` o cualquier `while` (así los traduce un compilador a código de máquina). Y sin embargo, ¿cuántos de ustedes escriben `goto` a mano? Ninguno. Y sigan así. La razón no es que `if/while` hagan algo que `goto` no pueda, es al revés: son *menos* poderosos a propósito. Atan el salto a la condición que lo justifica, en un solo lugar, y hacen imposible olvidarse de la etiqueta o dejar un salto colgado. Se acuerden de assembler con los `JMP`. Flashbacks de Vietnam.

Un semáforo es el `goto` de la sincronización. `acquire()` y `release()` son operaciones sueltas, no atadas entre sí ni a ningún dato. Nada impide tomar un semáforo y liberar otro por error, olvidarse de liberar (deadlock), o liberar sin haber tomado (invariante roto). Y como vimos en `CaminoUnaViaFIFO`, un problema mediano ya tiene cinco semáforos distintos desparramados por la clase: la sincronización no está atada a los datos que protege.

Un monitor tiene con `if/while` la misma relación que estos tienen con `goto`: no resuelve nada que un semáforo no pudiera resolver ya (en la próxima sección construimos uno a mano, con semáforos). Lo que da es disciplina, separando dos cosas que un semáforo contador mezcla en una sola:

1. **Exclusión mutua** sobre los datos privados: quién puede tocarlos ahora. Automática, garantizada una sola vez en el borde del objeto, imposible de olvidar.
2. **Bloqueo condicional**: esperar a que se cumpla una condición. Explícito, separado del lock, escrito como una condición booleana sobre variables reales, no como un número que sube y baja.

Con un semáforo contador estas dos cosas están fusionadas: el número es a la vez “cuántos pueden pasar” y “quién espera a que sea mayor a cero”. Un monitor las separa: quién toca los datos ahora (el lock implícito) queda desacoplado de qué condición estoy esperando (una variable de condición explícita). Esa separación es lo que vamos a construir. Pero primero, un repaso rápido de la teórica, para tener el vocabulario a mano.

3. Qué es un monitor (repaso)

Un monitor (Tony Hoare, 1974) combina tipo de dato abstracto y exclusión mutua: hay un monitor por objeto o grupo de objetos relacionados, con todos sus campos privados. Dentro del mismo monitor, las operaciones se ejecutan en exclusión mutua entre sí; entre monitores distintos, se entrelazan libremente.

Los elementos de un monitor son tres:

- Un conjunto de operaciones encapsuladas en la clase.
- Un único lock global, automático, que asegura la exclusión mutua.
- Variables de condición para la sincronización condicional adicional (“el buffer está lleno”, “todavía no llegaron los *n* porteños de esta ronda”).

Una variable de condición es una cola FIFO de threads bloqueados, manejada por la abstracción: no almacena valores, sólo procesos suspendidos. Tiene dos operaciones: **wait(c)** bloquea al proceso y **libera el lock** como parte atómica de bloquearse (si no lo liberara, nadie podría entrar a cambiar lo que se espera, deadlock consigo mismo). **signal(c)** despierta al primero bloqueado en *c*, y no hace nada si la cola está vacía (a diferencia de un semáforo, no queda “guardado” para el próximo).

Como para repetir, porque para nosotros también fue confuso la primera vez que lo aprendimos. **Una variable de condición es una cola de procesos bloqueados**. Sí, se llama variable y es una cola. No cuestionen.

Esto es el punto anterior hecho concreto: **wait(c)** no chequea ningún contador, siempre bloquea. Quien decide si hay que bloquearse es el programador, con un **if/while** explícito alrededor del **wait**. Con un semáforo, esa decisión está escondida adentro de **acquire()**.

4. Construyendo un monitor a mano: MonitorCasero

Manos limpias: manos a la obra. Presten especial atención a esta parte.

La buena noticia es que ya tenemos el ingrediente principal, sin darnos cuenta: la cola de semáforos privados de **CaminoUnaViaFIFO**. Generalizándola (que crezca y decrezca, sin tamaño fijo) y combiniándola con un único mutex, tenemos un monitor genérico y reutilizable, construido con semáforos, sin tocar **synchronized/wait/notify**.

En pseudocódigo:

```
clase MonitorCasero:
    mutex = Semaphore(1)
    esperando = cola vacia (de semaforos)

    entrar():
        mutex.acquire()

    salir():
        mutex.release()

    esperar():
```



```

miTurno = Semaphore(0)
agregar miTurno al final de esperando
mutex.release()      // suelto el lock, como wait()
miTurno.acquire()    // me bloqueo en MI semaforo, no en uno compartido
mutex.acquire()      // al despertar, vuelvo a competir por el lock

avisar():
    si esperando no esta vacia:
        sacar el primero de esperando y liberarlo (release)

avisarATodos():
    mientras esperando no esta vacia:
        sacar el primero y liberarlo (release)

```

Y la versión en Java, línea por línea igual al pseudocódigo:

```

public class MonitorCasero {
    private final Semaphore mutex = new Semaphore(1);
    private final Deque<Semaphore> esperando = new ArrayDeque<>();

    public void entrar() throws InterruptedException {
        mutex.acquire();
    }

    public void salir() {
        mutex.release();
    }

    public void esperar() throws InterruptedException {
        Semaphore miTurno = new Semaphore(0);
        esperando.addLast(miTurno);
        mutex.release();
        miTurno.acquire();
        mutex.acquire();
    }

    public void avisar() {
        if (!esperando.isEmpty()) {
            esperando.removeFirst().release();
        }
    }

    public void avisarATodos() {
        while (!esperando.isEmpty()) {
            esperando.removeFirst().release();
        }
    }
}

```

Por qué funciona:

- **Exclusión mutua:** mutex es un semáforo binario y sólo entrar()/salir() lo tocan. A lo sumo un thread está “adentro” en cualquier momento.

- **El orden en esperar() importa:** primero se libera `mutex`, después se bloquea en `miTurno`. Al revés, nadie podría entrar a llamar `avisar()`, deadlock consigo mismo. Es la misma razón por la que `wait(c)` exige liberar el lock como parte atómica de bloquearse.
- **Un semáforo privado por thread, no uno compartido:** si esperando fuera un único semáforo contador, un `release()` de `avisar()` lo podría tomar cualquiera, incluso alguien que recién llega. Es el mismo bug que `exitQueue` evita en `CaminoUnaViaFIFO`: un permiso apuntado a alguien sólo despierta a esa persona si viaja por un semáforo que nadie más comparte.

Nota. `avisar()` no entrega el mutex a quien despierta, sólo libera `miTurno`. El mutex sigue en manos de quien llamó a `avisar()` hasta su propio `salir()`; el despertado vuelve a pelear el mutex desde cero con `mutex.acquire()`, en igualdad de condiciones con cualquiera que llame a `entrar()` en ese instante. Es la disciplina *Signal y Continúa*, la misma que usa Java (Sección 8).

5. The Porteños strike back

Con nuestro `MonitorCasero`, probémoslo con un problema que ya conocemos. Una barrera junta a `n` threads en un punto de encuentro: nadie sigue hasta que los `n` llegaron. El problema de los porteños que suben el volcán Lanín (que subían en tramos, con la excursión esperando en cada pirca antes de arrancar el tramo siguiente) ya lo resolvimos con semáforos.

5.1. Repaso: Barrera con semáforos

```
public void esperar() throws InterruptedException {
    mutex.acquire();
    count++;
    if (count == n) {
        turnstile.release(n);
    }
    mutex.release();
    turnstile.acquire();

    mutex.acquire();
    count--;
    if (count == 0) {
        turnstile2.release(n);
    }
    mutex.release();
    turnstile2.acquire();
}
```

El detalle molesto: la barrera se usa en loop, ronda tras ronda. Hay que evitar que un thread rápido, ya cruzado `turnstile`, entre a la ronda siguiente antes de que los demás terminen de salir de la anterior. `turnstile2` existe sólo para eso: no deja pasar a nadie hasta que la ronda actual se vació por completo. Sacarlo rompería la solución.

¿Se les ocurre la solución con nuestro monitor casero? Piénsenlo un ratito. ¿Qué partes pasarían a ser parte del monitor, y qué partes seguirían siendo responsabilidad de la barrera? ¿Qué datos privados necesita la barrera para funcionar?

5.2. Porteños monitoreados



```
public void esperar() throws InterruptedException {
    monitor.entrar();

    int miFase = fase;
    count++;
    if (count == n) {
        count = 0;
        fase++;
        monitor.avisarATodos();
    } else {
        while (fase == miFase) {
            monitor.esperar();
        }
    }

    monitor.salir();
}
```

Un solo objeto (`monitor`) en vez de tres semáforos a mano. El segundo molinete pasó a ser una comparación de enteros: `fase == miFase`. El último en llegar reinicia `count` y avanza `fase`; los que esperaban guardaron su `miFase` antes de bloquearse y comparan contra ese valor al volver, saliendo del `while` justo cuando la ronda cambió. Sin `fase`, un thread rápido de la ronda siguiente pisaría el `count` que todavía miran los rezagados: `fase` cumple el rol de `turnstile2`, como una condición legible en vez de un tercer semáforo.

Y fíjense lo que tenemos: datos privados (`count`, `fase`), exclusión mutua automática, y una forma de esperar sin busy-wait. Eso es, por definición, un monitor. No apareció de la nada: lo construimos con semáforos que ya conocíamos.

Pero ¿por qué un `while` y no un `if`? ¿No alcanza con preguntar una sola vez? Sigán leyendo...

6. Un monitor ideal: qué pasaría con if

¿Por qué no alcanza con preguntar una sola vez? Para responder esto hace falta pensar un momento en qué pasa justo después de un **signal/avisar**. Por un instante hay dos threads con pretensiones de estar “adentro” del monitor: quien avisó y quien se despertó. Como el monitor exige que a lo sumo uno ejecute a la vez, hace falta un orden entre tres grupos: entradas nuevas (E), recién despertados (W) y quien acaba de señalar (S).

La disciplina clásica de Hoare (1974), *Signal y Espera Urgente* o *IRR (Immediate Resumption Requirement)*, usa la prioridad $E < S < W$: el despertado corre **inmediatamente**, antes que cualquier otra cosa, incluso antes de que termine quien lo despertó. La consecuencia es fuerte: lo que era cierto al llamar a **signal** sigue siendo cierto cuando el despertado arranca, porque nadie más tuvo oportunidad de correr en el medio.

Bajo esa disciplina ideal, la Barrera se podría escribir así (pseudocódigo, esto no es Java real):

```
esperar():
    entrar()                // automatico
    miFase = fase
    count++
    if count == n:
        count = 0
        fase++
        avisarATodos()
    else:
        if (fase == miFase) // alcanza con "if", no hace falta "while"
            esperar()
    salir()
```

Ni siquiera haría falta **fase**: como nadie nuevo entra al monitor hasta que termine toda la cadena de despertados ($S < W$ sobre E), ningún thread liberado observa un **count** ya tocado por una ronda posterior. El if alcanza porque la certeza la garantiza la disciplina del monitor, no el código del programador.

Lindo, ¿no? Con esa disciplina hasta nos podríamos ahorrar **fase**. Pero antes de entusiasmarse: ningún lenguaje mainstream implementa IRR tal cual, es carísimo (colas de prioridad distintas para E, S y W, mutex perfectamente justo). Java, como casi todos los lenguajes modernos, eligió otra cosa. Vamos a ver cuál.

7. La versión real: synchronized, wait, notifyAll

Ahora que construimos el nuestro a mano, miremos qué hace Java, sabiendo exactamente qué esperar.

```
public synchronized void esperar() throws InterruptedException {
    int miFase = fase;
    count++;
    if (count == n) {
        count = 0;
        fase++;
        notifyAll();
    } else {
```

```

        while (fase == miFase) {
            wait();
        }
    }
}

```

La correspondencia con `MonitorCasero` es directa:

MonitorCasero	Java
entrar() / salir()	synchronized
esperar()	wait()
avisarATodos()	notifyAll()

Cada objeto de Java trae un lock intrínseco y exactamente una cola de condición implícita: lo mismo que armamos a mano con `mutex` y `esperando`, la JVM ya lleva la cuenta. `synchronized` es *azúcar sintáctico*¹ para `entrar()/salir()` sobre ese objeto, con una diferencia importante: es reentrante. Un thread que ya tiene el lock puede volver a entrar a otro bloque `synchronized` sobre el mismo objeto sin bloquearse a sí mismo, algo que nuestro `mutex.acquire()` casero no soporta. `wait()/notify()/notifyAll()` sólo tienen sentido dentro de un `synchronized` sobre ese objeto (si no, `IllegalMonitorStateException`), por la misma razón que en `MonitorCasero`: necesitan el lock tomado para liberarlo de forma atómica al bloquearse.

Java sólo da una cola de condición por objeto, la misma limitación que `esperando`. Si un monitor necesita distinguir varias condiciones, `notifyAll()` despierta a todo el mundo aunque casi nadie tenga algo para hacer (correcto gracias al `while`, pero desperdicia trabajo). Para eso están `ReentrantLock` y `Condition`, con varias colas explícitas (Sección 9).

8. Por qué no hay certeza: `while`, fases y *spurious wakeups*

Volvamos entonces a la pregunta que dejamos pendiente: ¿por qué la Barrera necesita `while` y no `if`? Java (Lampson y Redell, 1980) implementa *Signal y Continúa*, con prioridad $E = W < S$: quien llama a `notify/notifyAll` **sigue corriendo**, no suspende nada ni entrega el lock. El despertado sólo pasa a estar “listo” y compite por el lock en igualdad con cualquier entrada nueva. No es una elección arbitraria: es exactamente lo que `MonitorCasero` hace naturalmente, ya que `avisar()` libera `miTurno` pero el `mutex` sigue en manos de quien avisó hasta su propio `salir()`. La disciplina de Java sale directamente de construir un monitor con semáforos comunes, sin mecanismo especial de prioridad.

La consecuencia: entre que un thread se despierta y vuelve a correr puede pasar cualquier cosa, incluida otra ronda entera de la barrera. Por eso hace falta `fase`: es la única forma que tiene el despertado de saber si la ronda que le prometieron sigue siendo la actual. Por eso el `if` de la Sección 6 deja de ser válido y hace falta `while (fase == miFase) wait();`: no es cautela de más, es la única opción correcta sin IRR.

¹Ohlalá señor francés

8.1. Spurious wakeups, en profundidad

Lo anterior explica por qué un `wait()` puede volver *tarde*, después de que la condición ya cambió de nuevo. Hay una segunda razón, independiente, para que `while` sea obligatorio: un `wait()` puede volver sin que **nadie, nunca, haya llamado** `notify()` ni `notifyAll()`. Es el *spurious wakeup* (despertar espurio), y conviene separarlo de la discusión anterior: son dos fenómenos con causas distintas, que casualmente se arreglan con el mismo `while`.

- **Causa 1 (la de las secciones anteriores): orden.** Hubo un `notify/notifyAll` real, apuntado a una condición real, pero como Java no da IRR, para cuando el thread despertado efectivamente vuelve a correr, algún otro thread ya modificó el estado y la condición dejó de valer. El wakeup fue legítimo, lo que falló fue la certeza temporal.
- **Causa 2 (la de esta sección): ausencia total de causa.** El thread vuelve de `wait()` sin que haya existido ningún `notify` en absoluto, ni tarde ni a tiempo. No hay ningún evento del programa que lo explique.

La especificación de `Object.wait()` permite la Causa 2 explícitamente, no es un bug de la JVM: el propio Javadoc dice que un thread se puede despertar sin haber sido notificado, interrumpido, ni haber vencido ningún timeout, y que el código tiene la obligación de revisar la condición real y volver a esperar si no se cumple. La garantía que Java ofrece sobre `wait()` no es “vuelvo sólo si alguien me notificó”, es apenas “en algún momento puedo volver”. Todo lo demás es responsabilidad del programador.

¿Por qué esta laxitud a propósito? Porque `wait()/notify()` no están implementados desde cero: en la JVM sobre Linux, el mismo camino de `Semaphore.acquire()` visto en la anterior clase practica de semáforos (`acquire()` → `AbstractQueuedSynchronizer` → `LockSupport.park()` → `pthread_cond_wait` con un `futex` debajo) es, con variantes, el que usa `Object.wait()`. Y POSIX permite explícitamente que `pthread_cond_wait` retorne de forma espuria, por razones legítimas:

- **Señales del sistema operativo.** Una señal (la que usa el recolector de basura para pausar threads en un *safe point*, u otra cualquiera) puede interrumpir el syscall de espera y hacerlo retornar, sin relación con la condición que el thread miraba.
- **Optimizaciones del kernel.** Los `futex` de Linux agrupan threads según un *hash* de la dirección de memoria, no según la condición exacta; dos condiciones distintas pueden colisionar en el mismo balde y despertar a quien no correspondía.
- **Evitar el “malón” (*thundering herd*).** Algunas implementaciones prefieren despertar de más y dejar que cada thread se fije si le tocaba, en vez de coordinar un despertar perfectamente selectivo.

Nada de esto tiene que ver con nuestra Barrera ni con `count`: son razones de la capa de sistema operativo. Java no promete filtrarlas (sería carísimo o imposible), y le traslada la responsabilidad al programador con una regla simple.

Un ejemplo mínimo, para ver que no es un problema de concurrencia entre threads. Un único thread consumidor de un buffer, sin ningún productor corriendo todavía:

```
public synchronized int leer() throws InterruptedException {
    if (estaVacio) {          // mal: "if" en vez de "while"
        wait();
    }
}
```

```
    estaVacio = true;
    return buf;           // valor incorrecto si wait() volvió espurio
}
```

Si `wait()` retorna de forma espuria (nadie llamó `notify()`, `estaVacio` sigue en `true`), el `if` ya se evaluó una vez y no se vuelve a mirar: el método sigue, pisa `estaVacio` y devuelve un `buf` que nunca fue escrito. No hace falta ningún otro thread compitiendo por el lock, alcanza con que el sistema operativo despierte al thread una vez, sin motivo. Con `while (estaVacio) wait();` el thread reevalúa, encuentra `true` de nuevo, y se vuelve a bloquear sin efecto observable: el despertar espurio queda absorbido por el chequeo.

Las dos causas son independientes. Si Java implementara IRR (Causa 1 desaparece, un `if` alcanzaría para el orden), la Causa 2 seguiría intacta: el sistema operativo podría seguir despertando threads sin aviso. Y al revés: un lenguaje sin despertares espurios pero con Signal-y-Continúa seguiría necesitando `while` por la Causa 1. `while` no corrige un único problema, corrige dos fuentes de incertidumbre distintas que en Java están las dos presentes a la vez.

La regla práctica: volver de `wait()` no transmite ninguna información, no dice “la condición es verdadera” ni “algo cambió”. Lo único correcto es releer las variables reales (`fase`, `count`, `estaVacio`) y decidir desde cero. Tratar el regreso de `wait()` como un evento con significado propio es exactamente el error que el `while` previene. Es una regla tan grave de romper que herramientas de análisis estático (como ErrorProne, de Google) marcan como error un `wait()` fuera de un `while`: no es pedantería, es un bug pattern real.

La regla general vale siempre, en Java y en cualquier lenguaje con monitores reales (C con `pthread`s, C++, C#): `wait()` (o `monitor.esperar()`) va siempre adentro de un `while (!condicion)`, nunca de un `if`. La versión con `if` sólo es correcta como experimento mental para un monitor con IRR, una idealización de manual que ni `synchronized` da, ni alcanzaría si además hubiera despertares espurios (que IRR, por definición, no contempla: describe el orden dentro del monitor, no el sistema operativo debajo).

9. Elecciones del CoDep

Cambiemos de problema. Hasta acá vimos monitores con una sola condición de espera. ¿Qué pasa cuando hace falta más de una?

9.1. El problema

Es la semana de elecciones para renovar a los representantes estudiantiles en el Consejo Departamental (CoDep) de Computación. El flujo de votantes se concentra frente a la oficina de la secretaria del DC, que dispone de:

- Una única **cabina de votación**, para emitir el sufragio en privado.
- Una **Secretaria** del departamento, que valida la identidad y el padrón de cada estudiante antes de habilitarle la cabina.
- Una **fila** en el pasillo, con capacidad estricta para N personas esperando.

La **Secretaria** funciona así: si no hay nadie en la fila ni nadie votando, descansa hasta que llegue un votante. Cuando un estudiante se presenta, lo atiende, verifica el padrón y le habilita la cabina. Mientras el estudiante vota, la Secretaria espera a que salga, deposite el sobre y desocupe el circuito, recién ahí atiende al siguiente².

El **estudiante**, al llegar, evalúa la fila: si ya hay N personas, le da fiaca esperar tanto y se va (a cursar, o a Maximia, o al kiosco del CECEN) sin votar. Si hay lugar, se suma al final. Cuando la Secretaria lo llama, pasa a la ventanilla, entra a la cabina, vota, deposita el sobre y se retira.

Antes de avanzar, tómense 5 minutos para entender el problema. ¿Cómo lo resolverían con semáforos? ¿Cuántos semáforos harían falta? ¿Les hace acordar a algún otro problema que hayamos visto?

9.2. Diseñando el monitor, en pseudocódigo (semántica Mesa)

Conviene diseñar el monitor en pseudocódigo antes de escribir Java, ya con semántica Mesa (**while**, nunca **if**), con la notación de la teórica: **wait(c)/signal(c)** sobre condiciones con nombre.

¿Cuántas condiciones hacen falta? Al menos dos, porque la Secretaria espera por dos motivos distintos que mezclados en una sola condición serían confusos: que llegue alguien (**hayLlegada**), o que termine el votante actual (**terminoVoto**). Los estudiantes en fila necesitan una tercera, para saber cuándo les toca (**puedeEntrar**).

Ojo con el orden de atención: si sólo guardáramos un contador de gente en la fila y la Secretaria hiciera **signal(puedeEntrar)** a secas, Mesa no garantiza que despierte al que llegó primero (ni **signal**, sin orden garantizado en el peor caso, ni **signalAll**, donde todos compiten en igualdad). Hace falta el mismo truco que **fase** en la Barrera: cada estudiante saca un ticket al llegar, y sólo avanza cuando la Secretaria anuncia ese ticket explícitamente.

```
monitor Secretaria {
    int enSecretaria = 0    // esperando + votando, tope N+1
    int proximoTicket = 0   // ticket que le toca al proximo en llegar
    int llamando = -1      // ticket que se esta atendiendo ahora mismo
    boolean votoDepositado = false

    condition hayLlegada    // la espera la Secretaria: llego alguien nuevo
    condition terminoVoto   // la espera la Secretaria: el votante actual termino
    condition puedeEntrar   // la esperan los estudiantes en la fila: les toca su ticket

    // el estudiante llama a esto al llegar
    int pedirTurno() {
        if (enSecretaria == N + 1)
            return -1          // sala llena, se cansa y se va
        enSecretaria++
        miTicket = proximoTicket++
        signal(hayLlegada)
        return miTicket
    }

    // el estudiante, con su ticket, espera a que lo llamen
    void esperarLlamado(miTicket) {
        while (llamando != miTicket)
```

²¿Y después le corta el pelo? Ah no, nos confundimos de problema


```

        wait(puedeEntrar)
    }

    // el estudiante avisa despues de depositar el sobre
    void avisarVotoDepositado() {
        votoDepositado = true
        signal(terminoVoto)
    }

    void salir() {
        enSecretaria--
    }

    // la Secretaria, en loop, llama a esto una vez por votante
    void atenderSiguiente() {
        while (llamando + 1 >= proximoTicket)
            wait(hayLlegada)          // todavia no llego nadie nuevo
        llamando++
        signalAll(puedeEntrar)        // puede haber varios en la fila, cada uno chequea su
        ticket

        votoDepositado = false
        while (!votoDepositado)
            wait(terminoVoto)
    }
}

```

`signal(hayLlegada)` y `signal(terminoVoto)` usan `signal` simple (a lo sumo un thread los espera, la Secretaria); `puedeEntrar` usa `signalAll` porque puede haber varios estudiantes en la fila y sólo `while (llamando != miTicket)` filtra al que le toca.

9.3. Pasando a Java con un único monitor: el problema de synchronized

Acá aparece la dificultad: `synchronized` da un lock intrínseco y **una sola** cola de condición por objeto. Nuestro diseño pide tres condiciones lógicas, así que las tres (`hayLlegada`, `terminoVoto`, `puedeEntrar`) terminan compartiendo la misma cola física.

La tentación es usar `notify()` pensando en apuntarle a quien corresponde, para no pagar el costo de despertar a todos. Pero `notify()` no elige por vos: despierta a algún thread de la cola, sin noción de “para qué” esperaba cada uno. Puede fallar así:

1. Un estudiante A (ticket 0) está en la fila, bloqueado en la única cola implícita (lógicamente esperando `puedeEntrar`).
2. La Secretaria también está bloqueada en esa misma cola (lógicamente esperando `hayLlegada`).
3. Llega un estudiante B, incrementa `proximoTicket` y llama a `notify()`, con la intención de despertar a la Secretaria.
4. Java despierta a **alguien** de la cola, sin garantía de cuál. Supongamos que despierta a A.
5. A vuelve a evaluar su `while (llamando != miTicket)`: sigue siendo cierto (todavía nadie lo llamó), así que se vuelve a bloquear.

6. La Secretaria sigue dormida. El único `notify()` disponible ya se gastó en A, que no podía hacer nada con él.

Desde el punto de vista de la Secretaria, ese aviso se perdió: hubo un evento real y un `notify()` real, pero despertó a quien no correspondía, y quien lo necesitaba se quedó esperando hasta que otro `notify()` no relacionado la despierte por casualidad. Es un despertar perdido por *multiplexado*: varias condiciones lógicas en una sola cola física hacen que un `signal` se pueda malgastar en un thread que esperaba otra cosa.

La única forma segura de usar un solo `synchronized` con varias condiciones es no usar nunca `notify()`, sólo `notifyAll()`: despierta a todos los bloqueados en el objeto, cada uno reevalúa su `while` y sólo el que corresponde sigue, el resto se vuelve a bloquear sin efecto. Correcto, pero caro: cada `notifyAll()` paga por despertar a todo el mundo aunque casi siempre haya un único destinatario, un costo de $O(n)$ en cada aviso.

```
public synchronized int pedirTurno() {
    if (enSecretaria == N + 1) {
        return -1;
    }
    enSecretaria++;
    int miTicket = proximoTicket++;
    notifyAll(); // la Secretaria puede estar esperando a que llegue alguien
    return miTicket;
}

public synchronized void esperarLlamado(int miTicket) throws InterruptedException {
    while (llamando != miTicket) {
        wait();
    }
}

public synchronized void avisarVotoDepositado() {
    votoDepositado = true;
    notifyAll();
}

public synchronized void atenderSiguiente() throws InterruptedException {
    while (llamando + 1 >= proximoTicket) {
        wait();
    }
    llamando++;
    notifyAll(); // a la Secretaria misma no, pero a TODOS los que hacen fila

    votoDepositado = false;
    while (!votoDepositado) {
        wait();
    }
}
```

Funciona (cada `while` filtra correctamente), pero cada una de las cuatro operaciones que avisan algo despierta a la sala entera, sin distinguir a quién le interesaba.

9.4. ReentrantLock y Condition: una cola por cada motivo de espera

`java.util.concurrent.locks` da la pieza que falta: un lock explícito (`Lock`, típicamente `ReentrantLock`) del que se piden variables de condición independientes con `lock.newCondition()`, cada una con su propia cola FIFO. La interfaz de `Condition` es casi calcada de `Object`, atada a una condición particular en vez de al objeto entero:

Object (una cola por objeto)	Condition (una cola por condición)
<code>wait()</code>	<code>await()</code>
<code>notify()</code>	<code>signal()</code>
<code>notifyAll()</code>	<code>signalAll()</code>

Un `ReentrantLock` más un puñado de `Condition` es la definición de monitor de la Sección 3 hecha realidad sin las limitaciones de `synchronized`: un lock (`lock.lock()/lock.unlock()`) y tantas colas con nombre como el problema necesite. El pseudocódigo del monitor `Secretaria` se traduce casi literal:

```
private final Lock lock = new ReentrantLock();
private final Condition hayLlegada = lock.newCondition(); // la espera la Secretaria
private final Condition terminoVoto = lock.newCondition(); // la espera la Secretaria
private final Condition puedeEntrar = lock.newCondition(); // la esperan los de la fila

public int pedirTurno() {
    lock.lock();
    try {
        if (enSecretaria == N + 1) {
            return -1;
        }
        enSecretaria++;
        int miTicket = proximoTicket++;
        hayLlegada.signal(); // solo la Secretaria puede estar esperando aca
        return miTicket;
    } finally {
        lock.unlock();
    }
}

public void esperarLlamado(int miTicket) throws InterruptedException {
    lock.lock();
    try {
        while (llamando != miTicket) {
            puedeEntrar.await();
        }
    } finally {
        lock.unlock();
    }
}

public void avisarVotoDepositado() {
    lock.lock();
    try {
        votoDepositado = true;
    }
```

```

        terminoVoto.signal(); // solo la Secretaria puede estar esperando aca
    } finally {
        lock.unlock();
    }
}

public void atenderSiguiente() throws InterruptedException {
    lock.lock();
    try {
        while (llamando + 1 >= proximoTicket) {
            hayLlegada.await();
        }
        llamando++;
        puedeEntrar.signalAll(); // puede haber varios en la fila, cada uno chequea su
        ticket

        votoDepositado = false;
        while (!votoDepositado) {
            terminoVoto.await();
        }
    } finally {
        lock.unlock();
    }
}

```

La ganancia es directa: `hayLlegada.signal()` y `terminoVoto.signal()` sólo pueden despertar a la Secretaria, porque nadie más se bloquea en esas colas, así que el bug de multiplexado desaparece por construcción. Queda un único `signalAll()` sobre `puedeEntrar`, que sigue costando $O(n)$ en la fila, pero eso ya es inevitable acá: cualquiera de la fila podría ser el del ticket siguiente, y n está acotado por N , no por el total de estudiantes del sistema. Para eliminar también ese costo haría falta una `Condition` por estudiante (la misma idea del array de semáforos de la Barrera), pero para este problema alcanza con lo anterior.

Nota. `unlock()` siempre en un bloque `finally`: a diferencia de `synchronized`, que libera el lock aunque el bloque termine por una excepción, con `ReentrantLock` esa liberación es manual. Olvidarla en un método que puede lanzar deja el lock tomado para siempre, el mismo bug que un `semaphore.release()` olvidado.

10. Aprobando esta materia: lapicera, cuaderno y coima

Un problema más, y con este cerramos la parte de monitores. Este es un clásico incómodo: la solución con semáforos sueltos, la que uno probaría primero de forma instintiva, directamente no funciona. Vamos a ver por qué, y cómo un monitor lo resuelve sin drama.

10.1. El problema

El curso se organizó y compraron al por mayor los materiales para aprobar la materia: para cada uno, una lapicera, un cuaderno con los apuntes completos, y un sobre con la guita juntada para

sobornar al docente³. Nadie es dueño de nada individualmente, es del curso. La hermana de uno es la tesorera: administra los materiales, pero nunca tiene las tres cosas prestables al mismo tiempo, porque lo que no circula prestado se está reponiendo. En el garage donde se guardan los materiales, en todo momento, hay como mucho dos de las tres, nunca las tres juntas, y no suelta nada nuevo hasta que alguien se lleva lo que había⁴.

Hay tres tipos de estudiante, que no dependen del stock para un elemento, pero sí recurren al stock para los otros dos:

- El estudiante **Cartuchera**: nunca se queda sin lapicera propia, pero jamás tuvo un cuaderno propio (fotocopia todo a último momento) y nunca puso un peso para sobornar a nadie⁵.
- El estudiante **Fotocopiadora**: siempre tiene el cuaderno con todos los apuntes, pero jamás una lapicera propia (las pierde todas) ni un peso puesto⁶.
- El estudiante **Forrado**: viene de familia con guita, nunca necesitó del stock para el soborno (paga papá), pero nunca se le ocurrió llevar lapicera ni cuaderno⁷.

Cuando en el garage aparecen exactamente las dos cosas que le faltan a alguno de los tres tipos, ese estudiante (y sólo ese) se acerca, las retira en préstamo, las suma a lo que ya traía y entra a rendir. La tesorera no vuelve a prestar nada hasta que esas dos cosas se retiren. ¿Cómo sincronizar a la tesorera y a los tres tipos de estudiantes para que cada uno tome lo que le corresponde, sin pisarse y sin esperar de más?

10.2. Por qué no alcanza con tres semáforos sueltos

La tentación es declarar un semáforo por elemento (**lapicera**, **cuaderno**, **soborno**, en 0), que la tesorera haga **release()** sobre los dos que corresponda, y cada tipo haga **acquire()** sobre los dos que le faltan. Se rompe enseguida: si la tesorera deja **cuaderno** y **soborno** (le tocaría al Cartuchera), nada impide que en ese instante el Fotocopiadora esté haciendo **soborno.acquire()** como parte de su propia espera, y se lo lleve sin que le sirva de nada sin la **lapicera** que todavía no está. El Cartuchera, que sí podía completar su combinación, queda bloqueado para siempre: el único permiso disponible lo agarró quien no lo iba a poder usar.

El problema de fondo es el de la Sección 2: un semáforo sólo prueba su propio contador, no puede preguntar de forma atómica “¿están disponibles estos dos OTROS recursos a la vez?”. Esa condición compuesta un monitor sí la escribe directo, como una expresión booleana sobre variables reales, bajo la misma exclusión mutua.

10.3. Diseño del monitor, en pseudocódigo (semántica Mesa)

Alcanza con tres booleanas y dos condiciones: una que esperan los tres tipos (**hayAlgo**) y otra que espera sólo la tesorera (**mostradorLibre**).

³Si hiciese falta. Aplican términos y condiciones. No incluye IVA.

⁴Salvo que alguno se quede fumando ahí. ¿Eh? ¿Qué nadie fuma? ¿Otra vez nos confundimos de problema?

⁵Dice que es por cuestiones morales. Pero todos sabemos que no tiene un mango, solo tiene lapiceras porque el tío tiene una librería en Caseros.

⁶Como buen encargado de fotocopiadora de esta universidad, se alimenta exclusivamente de pan relleno.

⁷Nunca hizo falta tomar nota en el Newman

```

monitor Mostrador {
    boolean lapicera = false, cuaderno = false, soborno = false

    condition hayAlgo          // la esperan los 3 tipos de estudiante
    condition mostradorLibre   // la espera la tesorera

    // la tesorera llama a esto cada vez que vuelve a tener algo prestable
    void reponer(a, b) {
        while (lapicera || cuaderno || soborno)
            wait(mostradorLibre) // todavia no se llevaron lo anterior
        marcar a y b como presentes
        signalAll(hayAlgo)
    }

    // cada estudiante llama a esto con los dos elementos que le faltan
    // (el tercero ya lo trae puesto, nunca le falta)
    void juntarElementos(faltanteA, faltanteB) {
        while (!(estaPresente(faltanteA) && estaPresente(faltanteB)))
            wait(hayAlgo)
        marcar faltanteA y faltanteB como ausentes
        signal(mostradorLibre)
    }
}

```

Vale el invariante “en todo momento hay 0 o 2 de los tres elementos presentes, nunca 1 ni 3”. Se cumple porque `reponer` sólo corre con los tres en `false` y pone exactamente dos en `true`, y `juntarElementos` sólo corre con sus dos en `true` y los pone en `false` juntos. Por eso, en cualquier instante hay a lo sumo un tipo cuya condición puede ser cierta: los otros dos, aunque el mismo `signalAll(hayAlgo)` los despierte, reevalúan su `while`, lo encuentran falso y se bloquean sin tocar nada. No hace falta ticket acá (a diferencia de la fila del CoDep, Sección 9): la combinación de elementos ya determina a quién le toca.

10.4. Código en Java

Con dos motivos de espera bien diferenciados, es una aplicación directa de `ReentrantLock` y `Condition` (con `synchronized` también funcionaría, al costo del mismo `notifyAll()` de la sección anterior, aunque acá el desperdicio es chico: nunca hay más de tres threads bloqueados).

```

static final int LAPICERA = 0, CUADERNO = 1, SOBORNO = 2;

private final Lock lock = new ReentrantLock();
private final Condition hayAlgo = lock.newCondition(); // la esperan los 3 tipos
private final Condition mostradorLibre = lock.newCondition(); // la espera la tesorera
private final boolean[] presente = new boolean[3];

public void reponer(int a, int b) throws InterruptedException {
    lock.lock();
    try {
        while (presente[LAPICERA] || presente[CUADERNO] || presente[SOBORNO]) {
            mostradorLibre.await();
        }
    }
}

```

```

        presente[a] = true;
        presente[b] = true;
        hayAlgo.signalAll();
    } finally {
        lock.unlock();
    }
}

public void juntarElementos(int faltanteA, int faltanteB) throws InterruptedException {
    lock.lock();
    try {
        while (!(presente[faltanteA] && presente[faltanteB])) {
            hayAlgo.await();
        }
        presente[faltanteA] = false;
        presente[faltanteB] = false;
        mostradorLibre.signal();
    } finally {
        lock.unlock();
    }
}

```

Cada tipo de estudiante es un thread en loop que llama a `juntarElementos` con su propio par fijo (Cartuchera: CUADERNO/SOBORNO; Fotocopiadora: LAPICERA/SOBORNO; Forrado: LAPICERA/CUADERNO), y la tesorera es un loop que, al quedarse sin nada prestable, elige al azar qué elemento deja sin reponer y llama a `reponer` con los otros dos:

```

int faltante = ThreadLocalRandom.current().nextInt(3); // el elemento que queda sin
reponer
mostrador.reponer((faltante + 1) % 3, (faltante + 2) % 3);

```

El archivo completo, con los tres threads de estudiante y el `main`, está en `Java/Parcial.java`.

11. Thread pool

Para cerrar la clase, cambiemos de tipo de problema. Todo lo anterior (colas de semáforos privados, `MonitorCasero`, `ReentrantLock/Condition`) resolvía threads coordinándose para acceder a un recurso compartido. Un *thread pool* es otra cosa: una herramienta de propósito general para no crear un `Thread` nuevo cada vez que hay algo para ejecutar en paralelo. Crear un thread real no es gratis (stack, registro en el scheduler del SO), así que para miles de tareas cortas conviene tener de antemano un puñado fijo de threads vivos y turnárselos. Y como van a ver, para construirlo alcanza con lo que ya sabemos.

11.1. Item a: el pool, con monitores

Enunciado. Implementar `ThreadPoolCasero`, que en su constructor recibe un entero `n` y arranca exactamente `n` threads (los *workers*). Debe ofrecer `submit(Runnable tarea)`, que encola una tarea para que la ejecute cualquier worker libre; si los `n` están ocupados, las tareas se acumulan hasta que

se libere alguno. No usar `ExecutorService` ni ninguna otra clase de `java.util.concurrent` de alto nivel: sólo `synchronized/wait/notify` (o `Lock/Condition`).

Es productor-consumidor: quien llama a `submit` produce tareas, los workers las consumen. La única diferencia con un buffer clásico es que acá lo que circula no son datos para leer, son pedazos de código para ejecutar. La cola es el único estado compartido que hay que proteger.

```
monitor Pool {
    queue<Runnable> tareas = vacia

    condition hayTareas

    // llamado por quien quiere que se ejecute una tarea
    void submit(tarea) {
        encolar tarea en tareas
        signal(hayTareas)    // alcanza avisar a uno: cualquier worker libre sirve
    }

    // llamado por cada worker, en loop infinito
    Runnable tomarTarea() {
        while (tareas esta vacia)
            wait(hayTareas)
        return desencolar de tareas
    }
}
```

Nada de esto es nuevo: `tomarTarea()` tiene la forma de cualquier `esperar()` de este apunte, `while` en vez de `if` por la misma razón (Sección 8): entre que un worker se despierta y vuelve a correr, otro puede haberle ganado de mano a la única tarea que había.

Lo que sí es nuevo: `tomarTarea()` sólo saca la tarea, **no la ejecuta**. Cada worker sale del monitor antes de correrla. Si la ejecución quedara adentro, la exclusión mutua obligaría a que un solo worker corra una tarea a la vez: los otros $n - 1$ quedarían bloqueados esperando el lock sólo para mirar la cola. Se perdería todo el sentido de tener n threads vivos.

```
private final Deque<Runnable> tareas = new ArrayDeque<>();

public ThreadPoolCasero(int n) {
    for (int i = 0; i < n; i++) {
        new Thread(this::loopDeWorker).start();
    }
}

// Alcanza con notify(): todos los workers esperan exactamente lo mismo
// (que haya UNA tarea cualquiera), asi que cualquiera de ellos sirve.
public synchronized void submit(Runnable tarea) {
    tareas.addLast(tarea);
    notify();
}

private void loopDeWorker() {
    while (true) {
        Runnable tarea;
        synchronized (this) {
```



```

        while (tareas.isEmpty()) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        tarea = tareas.removeFirst();
    }
    tarea.run(); // fuera del lock
}
}

```

Notar por qué acá `notify()` simple alcanza, a diferencia de las Elecciones del CoDep (Sección 9), donde un `notify()` mal dirigido podía despertar a un estudiante inútil y dejar a la Secretaria durmiendo. En el pool **todos los que esperan quieren exactamente lo mismo**: cualquier worker libre ejecuta cualquier tarea, no hay tickets ni tipos de espera para confundir. Cuando todos los waiters son intercambiables, `notify()` nunca despierta “al que no correspondía”. Guarden esta idea: en el Item b se da vuelta.

11.1.1.1. Haciéndolo más seguro: ¿qué pasa si una tarea explota?

El código de arriba tiene un problema que no se nota hasta que una tarea falla de verdad. `Runnable.run()` puede lanzar cualquier excepción no chequeada, y `tarea.run()` se llama sin ningún `try/catch` alrededor.

La excepción se escapa de `tarea.run()` y de `loopDeWorker()`. Java no reinicia un thread cuyo método principal terminó por una excepción no atrapada: el thread simplemente termina (después de imprimir el stack trace por `System.err`). El pool, que arrancó con `n` workers, se queda con `n - 1`. Si esto se repite, se va vaciando en silencio hasta quedarse sin ningún worker vivo, y las tareas nuevas se siguen encolando para siempre sin que nadie las ejecute. No hay ningún error visible, sólo tareas que dejan de correr.

La solución: atajar cualquier `Throwable` alrededor de la ejecución, *adentro* del loop del worker, para que una tarea que falla se lleve puesta sólo a sí misma.

```

try {
    tarea.run();
} catch (Throwable error) {
    System.err.println(Thread.currentThread().getName() + ": una tarea termino con una
    excepcion");
    error.printStackTrace();
    // seguimos: el worker vuelve al loop, no muere por una tarea que fallo
}

```

Dos detalles deliberados:

- **Se atrapa `Throwable`, no sólo `Exception`.** `Throwable` incluye a los `Error` (por ejemplo `StackOverflowError`). Un `catch (Exception e)` a secas no alcanzaría para protegernos de

todo lo que puede tirar una tarea arbitraria (aunque en un pool de producción vale discutir si conviene seguir corriendo después de un `Error` como `OutOfMemoryError`).

- **El catch está fuera del bloque `synchronized`.** Atraparla con el lock tomado no traería beneficio (la tarea ya se sacó de la cola) y alargaría innecesariamente cuánto tiempo está tomado el monitor.

Con este cambio, una tarea puede fallar todas las veces que quiera, que el pool sigue teniendo sus `n` workers disponibles.

11.2. Item b: tareas que devuelven un resultado, con promesas

Enunciado. Extender el pool para encolar tareas que devuelven un valor (`Callable<T>` en vez de `Runnable`). Quien encola una de estas tareas no se puede quedar bloqueado: tiene que poder seguir haciendo otras cosas, y más tarde pedir el resultado (bloqueándose ahí si no está listo), o decirle “avisame cuando termines”. El objeto que representa *un resultado que todavía no existe pero va a existir (o un error que puede pasar)* se llama una *promesa*.

11.2.1. Qué es exactamente una promesa

Una promesa es una caja vacía al crearse que, en algún momento futuro, una única vez, se llena: con un valor (si la tarea terminó bien) o con un error (si falló). Nadie sabe de antemano cuándo se va a llenar. Hay dos formas de interactuar con ella:

- **Preguntar y esperar** (`obtener()`): si ya está llena, devuelve el contenido ya mismo; si no, bloquea hasta que se llene.
- **Dejar dicho qué hacer** (`entonces(callback)`): no bloquea, deja un `callback` anotado. Si ya está llena, corre ya mismo; si no, corre más tarde, en el thread de quien la llenó.

Es lo que en JavaScript se llama *promise* y en Java `Future/CompletableFuture`: la misma idea, otro nombre. La vamos a llamar `PromiseCasera<T>`, y de nuevo la construimos antes de usar la de Java.

11.2.2. El monitor de una promesa

Una promesa es, otra vez, un monitor del tipo más chico posible: se resuelve una única vez (con éxito o error), y de ahí en más sólo se lee.

```
monitor PromiseCasera<T> {
    T valor
    Throwable error
    boolean listo = false
    list<Callback<T>> callbacksPendientes = vacia

    condition resuelta

    // llamado UNA vez, por quien ejecuto la tarea con exito
    void completar(v) {
        if (listo) return          // ya estaba resuelta: no se pisa
        valor = v
    }
}
```

```

    listo = true
    signalAll(resuelta)          // puede haber mas de uno esperando ESTE resultado
    para cada callback en callbacksPendientes: callback(v)
}

// llamado UNA vez, por quien vio fallar la tarea
void completarConError(e) {
    if (listo) return
    error = e
    listo = true
    signalAll(resuelta)
}

// bloquea hasta que este resuelta; si fue con error, lo relanza
T obtener() {
    while (!listo)
        wait(resuelta)
    if (error != null) relanzar error
    return valor
}

// NO bloquea: registra el callback, o lo corre ya mismo si ya esta lista
void entonces(callback) {
    if (listo && error == null)
        callback(valor)
    else if (!listo)
        agregar callback a callbacksPendientes
}
}

```

Cuatro decisiones para justificar:

1. **signalAll, no signal.** Puede haber más de un thread esperando exactamente esta promesa, y todos tienen que enterarse. Un **signal** simple despertaría a uno solo y dejaría al resto esperando para siempre. Es la contracara del pool: ahí los waiters eran intercambiables y **notify()** alcanzaba; acá todos quieren precisamente lo mismo.
2. **if (listo) return al principio de completar/completarConError.** Una promesa se resuelve una única vez: representa “lo que efectivamente pasó”, no “el último valor que alguien puso”. Sin esta guarda, una segunda llamada pisaría silenciosamente el resultado y quien ya hizo **obtener()** quedaría desincronizado con la promesa. Ignorar la segunda resolución es la misma elección de **CompletableFuture**.
3. **Los errores no disparan los callbacks de entonces.** Un callback de **entonces** espera un valor, no tiene qué hacer si la tarea falló. Para reaccionar a un error se usa **obtener()**, que lo relanza. Un **PromiseCasera** más completo tendría también un **exceptionally**, no hace falta acá.
4. **entonces nunca bloquea a quien lo llama.** Si la promesa no está lista, sólo anota el callback y devuelve el control. El callback corre en el thread que llame a **completar**, un worker del pool, así que nunca debería hacer algo lento: tataría a ese worker, el mismo problema que si **tarea.run()** corriera adentro del monitor del pool.

```

public class PromiseCasera<T> {
    private T valor;
    private Throwable error;
    private boolean listo = false;
    private final List<Consumer<T>> callbacks = new ArrayList<>();

    public void completar(T valor) {
        List<Consumer<T>> aEjecutar;
        synchronized (this) {
            if (listo) {
                return; // ya estaba resuelta: no se pisa
            }
            this.valor = valor;
            this.listo = true;
            notifyAll();
            aEjecutar = new ArrayList<>(callbacks);
            callbacks.clear();
        }
        // fuera del lock, misma razon que tarea.run() en el pool
        for (Consumer<T> callback : aEjecutar) {
            callback.accept(valor);
        }
    }

    public synchronized void completarConError(Throwable error) {
        if (listo) {
            return;
        }
        this.error = error;
        this.listo = true;
        notifyAll();
    }

    public synchronized T obtener() throws InterruptedException, ExecutionException {
        while (!listo) {
            wait();
        }
        if (error != null) {
            throw new ExecutionException(error);
        }
        return valor;
    }

    public void entonces(Consumer<T> callback) {
        T valorListo = null;
        boolean yaListoConExito;
        synchronized (this) {
            yaListoConExito = listo && error == null;
            if (yaListoConExito) {
                valorListo = valor;
            } else if (!listo) {
                callbacks.add(callback);
            }
        }
    }
}

```

```

        if (yaListoConExito) {
            callback.accept(valorListo);
        }
    }
}

```

`completar` ejecuta los callbacks *fuera* del `synchronized`, copiando la lista antes de soltar el lock: la misma disciplina que `tarea.run()` en el pool. Si corrieran con el lock tomado, un callback lento bloquearía a cualquier otro thread que sólo quisiera `obtener()`.

11.2.3. Conectando el pool con la promesa

Con `PromiseCasera` armada, el segundo `submit` del pool reutiliza el primero: por dentro sigue siendo un `Runnable` como cualquier otro, que al terminar completa la promesa (con éxito o, ahora sí, con error, en vez de dejarla colgada).

```

public <T> PromiseCasera<T> submit(Callable<T> tarea) {
    PromiseCasera<T> promesa = new PromiseCasera<>();
    submit(() -> {
        try {
            promesa.completar(tarea.call());
        } catch (Exception e) {
            promesa.completarConError(e);
        }
    });
    return promesa;
}

```

También es una cuestión de seguridad: sin el `catch` que llama a `completarConError`, una tarea con resultado que falla dejaría su promesa sin resolver *para siempre*, y quien hiciera `obtener()` se bloquearía eternamente sin indicio de qué pasó. Con el `catch` de `loopDeWorker` como red adicional, el pool queda protegido en las dos puntas.

El archivo completo (pool y promesa, con un `main` que dispara una tarea que falla, una con resultado, y una con resultado que también falla) está en `Java/ThreadPoolCasero.java` y `Java/PromiseCasera.java`.

11.3. La versión que ya trae Java

`java.util.concurrent` provee las dos piezas, más completas: `Executors.newFixedThreadPool(n)` da un `ExecutorService` con el mismo pool de tamaño fijo (y su propio manejo cuidadoso de excepciones), y `submit(Callable<T>)` devuelve un `Future<T>` cuyo `get()` es nuestro `obtener()` (también re- lanza en un `ExecutionException`). `CompletableFuture` corresponde a `PromiseCasera`: `complete(v)` es `completar`, `completeExceptionally(e)` es `completarConError`, `thenAccept(callback)` es entonces. Pero va más lejos: encadena transformaciones sin bloquear en ningún punto (`thenApply`, `thenCompose`, `exceptionally`), y elige en qué executor corre cada paso. Esa parte queda para otra vuelta.

Y con esto llegamos al final. That's all folks! Repasando:

- Arrancamos con semáforos sueltos, vimos que la sincronización así queda desparramada por el código como un `goto`.
- Construimos con nuestras propias manos un monitor que ata la exclusión mutua a los datos y separa el bloqueo condicional en una condición explícita.
- Con ese monitor casero resolvimos la Barrera, entendimos por qué Java exige `while` en vez de `if` (dos veces: por la falta de IRR, y por los despertares espurios).
- Llevamos el monitor a problemas más grandes, con varias condiciones (CoDep), condiciones compuestas (el stock en el garage) y, para cerrar, una herramienta de propósito general (el thread pool) con su propia versión de promesas.

Esperamos que hayan entendido monitores: qué son, por qué usarlos, y qué problemas nos resuelven sus operaciones en Java: `synchronized`, `wait`, `notify` y `notifyAll`.